Lab 6: Nested Loops

Task 1: Insertion Sort (70pts)

Insertion sort is a simple sorting algorithm that iteratively inserts each element of an unsorted list into its correct position within the sorted portion of the list.

The algorithm of Insertion sort is described as follows:

Algorithm 1 Insertion Sort

Require: Array A of length n
Ensure: A is sorted in non-decreasing order

```
for i ← 2 to n do
    key ← A[i]
    j ← i − 1
    while j > 0 and A[j] > key do
        A[j + 1] ← A[j]
        j ← j − 1
    end while
    A[j + 1] ← key
end for
```

Implement the algorithm above using the RISC-V RV32I instruction set. The starter code, `insertion_sort.s` is provided.

Deliverables for Task 1:

- Your code as `lastname_insertion_sort.s`
- Screenshot of the sorted array in the memory tab after the code runs.

Task 2: Containment Graph (30pts)

Introduction

The containment graph is a directed graph that represents the relative positions of the nodes within an $m \times n$ grid. For each node in the network, with the following properties:

- If a node's x, y coordinates are both less than or equal to another graph (i.e., node f is within the region A of node e), then there is an edge connecting node f to node e. See Figure 1.
- Commutativity: If a node lies in region A of another node, then the other node must lie in region B of this node. (i.e., node g lies in the region B of node e, and node e lies in the region A of node e. See Figure 1.)
- Transitivity: If the first node lies in region A of the second node, and the second node lies in region A of the third node, then the first node lies in region A of the third node. (See node f, e, g in the figure 1.)

Programming Task and Test Cases

Design and implement an algorithm in RISC-V RV32I that reads an array of points and generates the containment graph as described above. The test cases and the resulting graphs are given in Figure 2 to Figure 5.

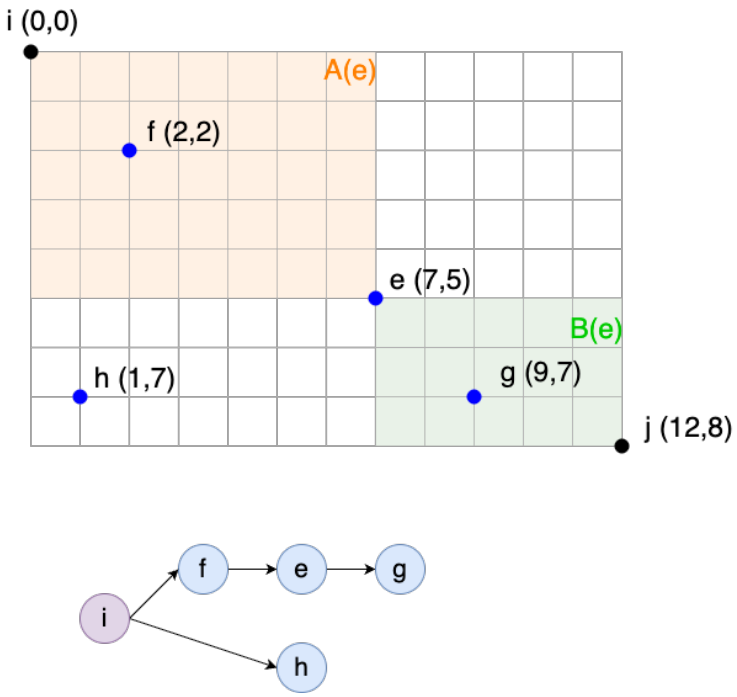

Figure 1: Demo Case

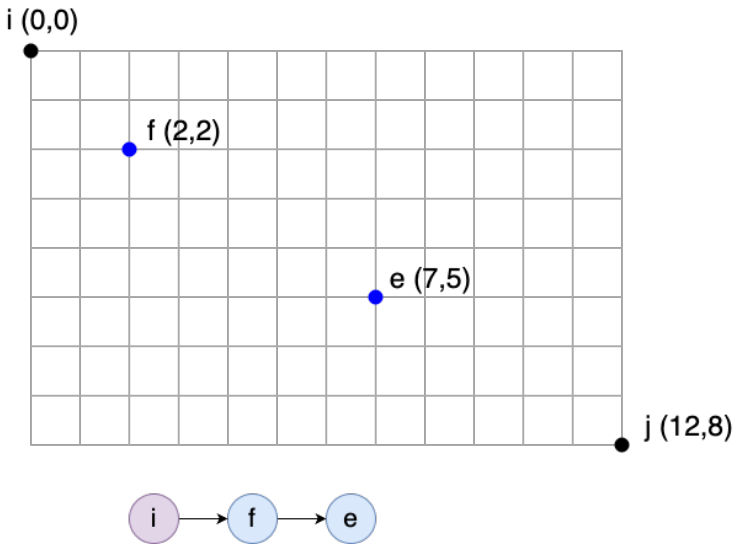

Figure 2: Case 1

Questions

- Can the containment graph become cyclic? Prove or disprove.
- What is the maximum number of branches that a node can have?

Deliverables for Task 2:

- Your code as `lastname_containment_graph.s`
- Screenshot of each case in the memory tab after the code runs.

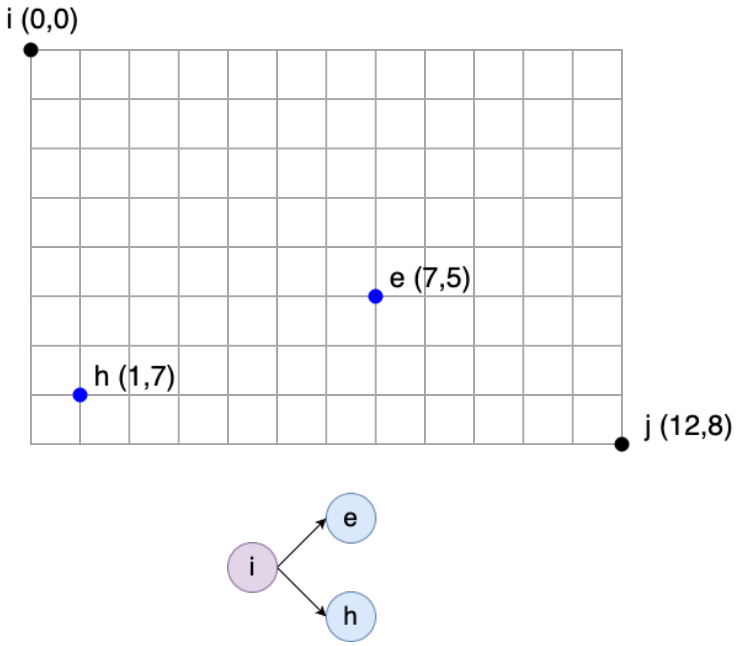

Figure 3: Case 2

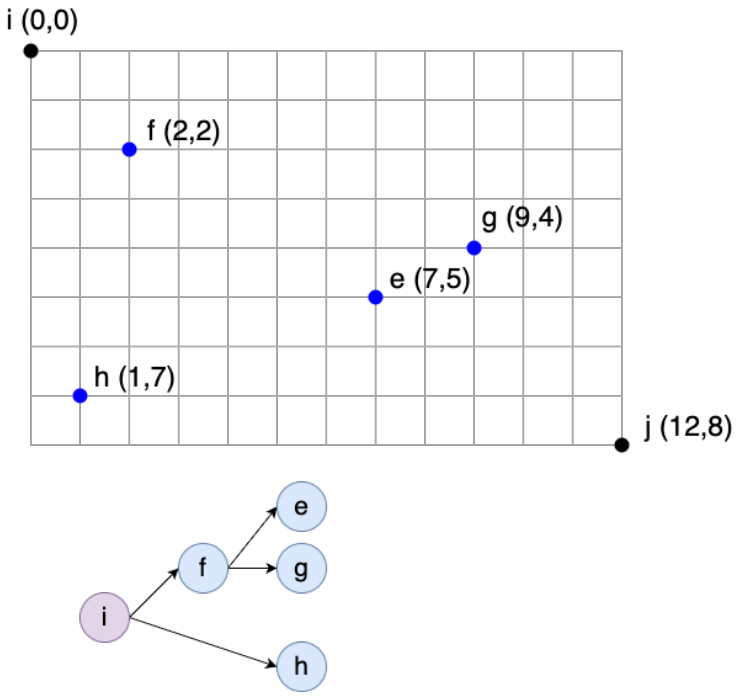

Figure 4: Case 3

- Your answers to the questions in `lastname_t2questions.s`

Tournament Round (50pts 1st place, 30pts 2nd place, 20pts 3rd place)

Demonstrate Task 2 in class. Your code should also include an additional module to traverse the graph and print the nodes in depth-first search order. The implementation that uses the least RAM wins the

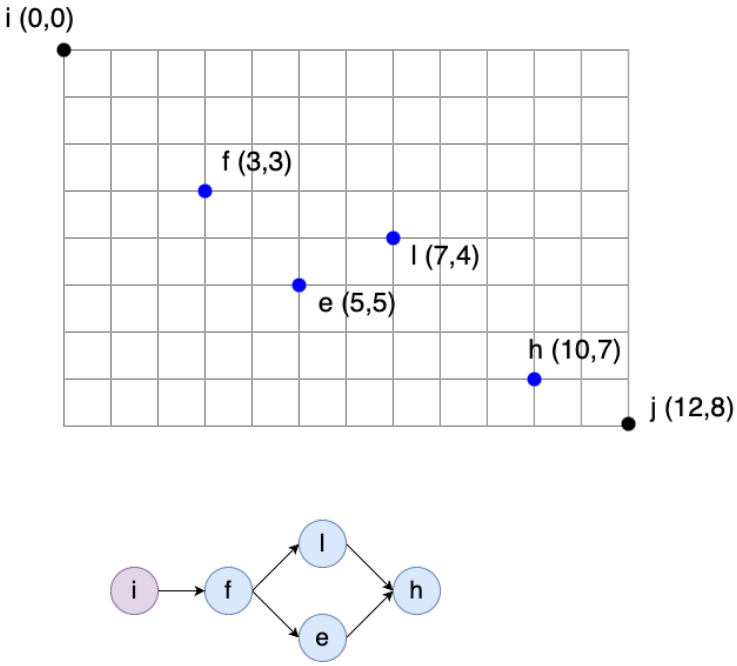

Figure 5: Case 4

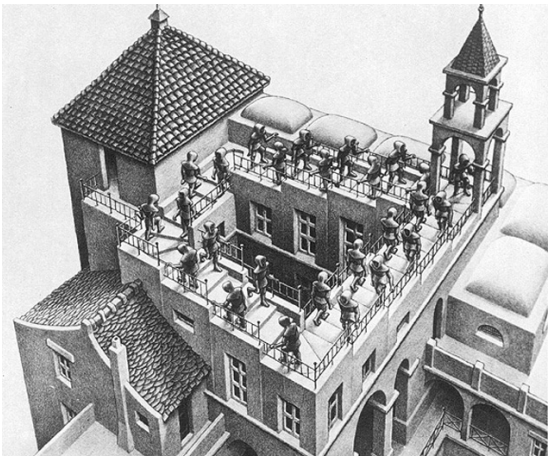

Figure 6: M.C. Escher (1898-1972), Upstairs and Downstairs,(1960; lithography) Reference `https://web.sbu.edu/theology/bychkov/escher_stairs.html`

tournament.